UWE Bristol | University of the West of England

Presentation by
Majid Mumtaz

Databases: DBs connectivity with Python

Week 10

Recap

GUI

Widgets

Event-Driven Programming

More on GUI with Python

OOP way to write Tkinter applications

Overview

UWE Bristol University of the West of England

Database Refresher

Databases with Python

Data Access Object

SqlAlchemy - Object Relational Mapper (ORM)

Model-View-Controller (MVC) Architecture

Working with Data and DB Connectivity

UWE Bristol University of the West of England

Have you thought about how will you preserve objects or object's states?

- *Database Systems: A practical approach to design, implementation, and management, Thomas Connolly and Carolyn Begg. Sixth Edition/Global Edition. (CB-GE)*

- *Fundamentals of Database Systems, Ramez Elmasri and Shamkant B Navathe. Seventh Edition/Global Edition. (EN-GE)*

Working with Data

- Data is one of the key entities in computing and an inseparable companion of programs.

- Flat data files consolidate data into a single plain-text format, such as CSV or JSON.
  - They are:
    - Useful when data is limited,
    - Readable and editable
    - Python libraries such as **pandas** use this formats easily
  - But, can it create difficulties to access and process when the size grows.

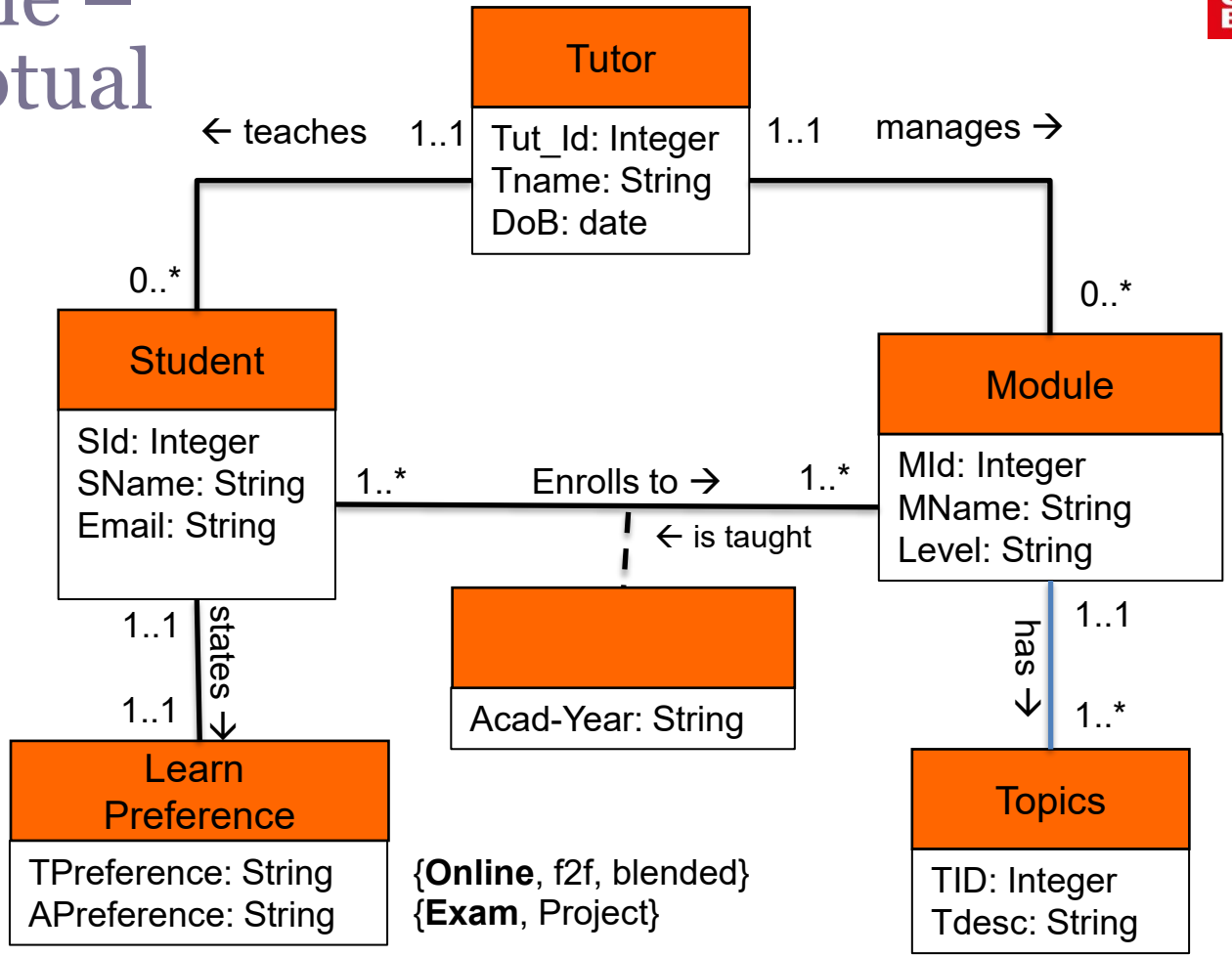
users.json > ...

```
[
    ["Zaheer Khan", "Prof", "3Q31", "zaheer2.khan@uwe.ac.uk"], ["Nathan Duran", "Senior Lecturer", "4Q60", "nathan.duran@uwe.ac.uk"], ["Barkha Javed", "Lecturer", "3E40", "barkha.javed@uwe.ac.uk"], ["M Khan", "Senior Lecturer", "4Q58", "mkhan@uwe.ac.uk"], ["Kun Wei", "Senior Lecturer", "2Q16", "kun.wei@uwe.ac.uk"], ["Majid Mumtaz", "Sinor Lecturer", "4Q58", "majid.mumtaz@uwe.ac.uk"]

]
```

Json file

  - Databases and Database Management Systems e.g., MySQL, Oracle, PostgreSQL

Example – Conceptual Model



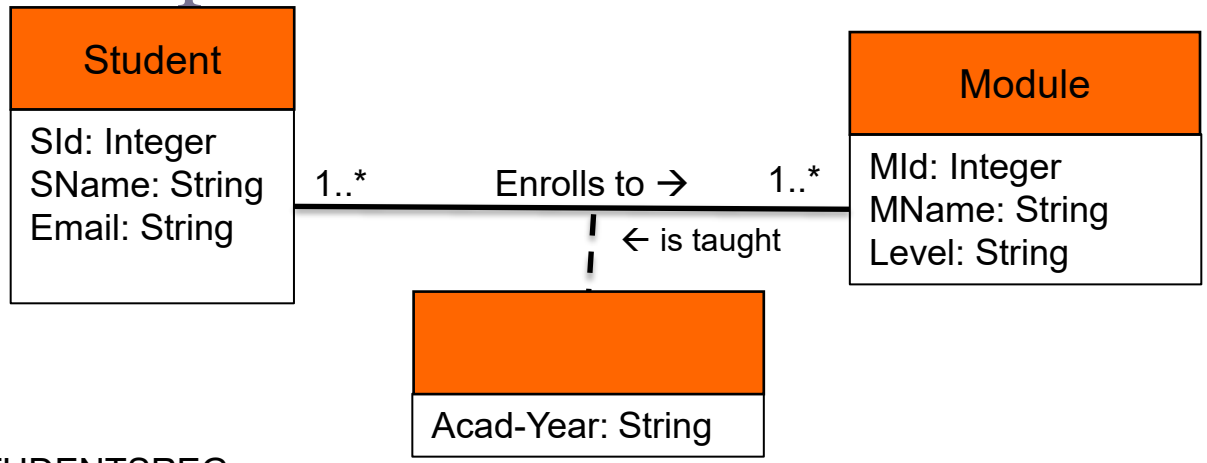
Build and validate logical data model

- Chapter 17 – CB-GE
- Table 17.2:
- Summarizes how to map entities and relationships to relations

Entity/Relationship	Mapping
Strong entity	Create relation that includes all simple attributes.
Weak entity	Create relation that includes all simple attributes (primary key still has to be identified after the relationship with each owner entity has been mapped).
1:* binary relationship	Post primary key of entity on 'one' side to act as foreign key in relation representing entity on 'many' side. Any attributes of relationship are also posted to 'many' side.
1:1 binary relationship:	
(a) Mandatory participation on both sides	Combine entities into one relation.
(b) Mandatory participation on one side	Post primary key of entity on 'optional' side to act as foreign key in relation representing entity on 'mandatory' side.
(c) Optional participation on both sides	Arbitrary without further information.
Superclass/subclass relationship	See Table 17.1
: binary relationship, complex relationship	Create a relation to represent the relationship and include any attributes of the relationship. Post a copy of the primary keys from each of the owner entities into the new relation to act as foreign keys.
Multi-valued attribute	Create a relation to represent the multi-valued attribute and post a copy of the primary key of the owner entity into the new relation to act as a foreign key.

And, don't forget Database Normalisation

Example – *:*



STUDENTSREG

STUDENT

SID	SNAME	EMAIL
1000	ABC	abc@abc.com
1001	DEF	def@def.com
1002	GHI	ghi@ghi.com
1003	JKL	jkl@jkl.com

Primary Key

MODULES

MID	MNAME	LEVEL
101	Web programming	1
102	Web Desgin	1
103	CMS	1
104	E-Commerce	1

Primary Key

Foreign Key

Foreign Key

SID	MID	ACAD_YEAR
1000	101	2014-2015
1000	102	2014-2015
1001	101	2014-2015
1001	103	2014-2015
1001	104	2014-2015
1002	104	2014-2015

STUDENT_ENROLEMENT

SQL – Simple example

UWE Bristol University of the West of England

STUDENTSREG

STUDENT

SID	SNAME	EMAIL
1000	ABC	abc@abc.com
1001	DEF	def@def.com
1002	GHI	ghi@ghi.com
1003	JKL	jkl@jkl.com

MODULES

MID	MNAME	LEVEL
101	Web programming	1
102	Web Desgin	1
103	CMS	1
104	E-Commerce	1

SID	MID	ACAD_YEAR
1000	101	2014-2015
1000	102	2014-2015
1001	101	2014-2015
1001	103	2014-2015
1001	104	2014-2015
1002	104	2014-2015

STUDENT_ENROLEMENT

```
CREATE DATABASE STUDENTSREG;
USE STUDENTSREG;
```

```
CREATE TABLE STUDENT
(
SID VARCHAR(10) NOT NULL,
PRIMARY KEY (SID),
SNAME VARCHAR(30),
EMAIL VARCHAR(50)
);
```

```
CREATE TABLE MODULES
(
MID VARCHAR(15) NOT NULL,
PRIMARY KEY (MID),
MNAME VARCHAR(40),
LEVEL VARCHAR(2)
);
```

```
CREATE TABLE STUDENT_ENROLEMENT
(
SID VARCHAR(10) NOT NULL,
FOREIGN KEY (SID) REFERENCES STUDENT(SID),
MID VARCHAR(15) NOT NULL,
FOREIGN KEY (MID) REFERENCES MODULES(MID),
ACAD_YEAR VARCHAR(10),
PRIMARY KEY (SID, MID, ACAD_YEAR)
);
```

SQL Commands

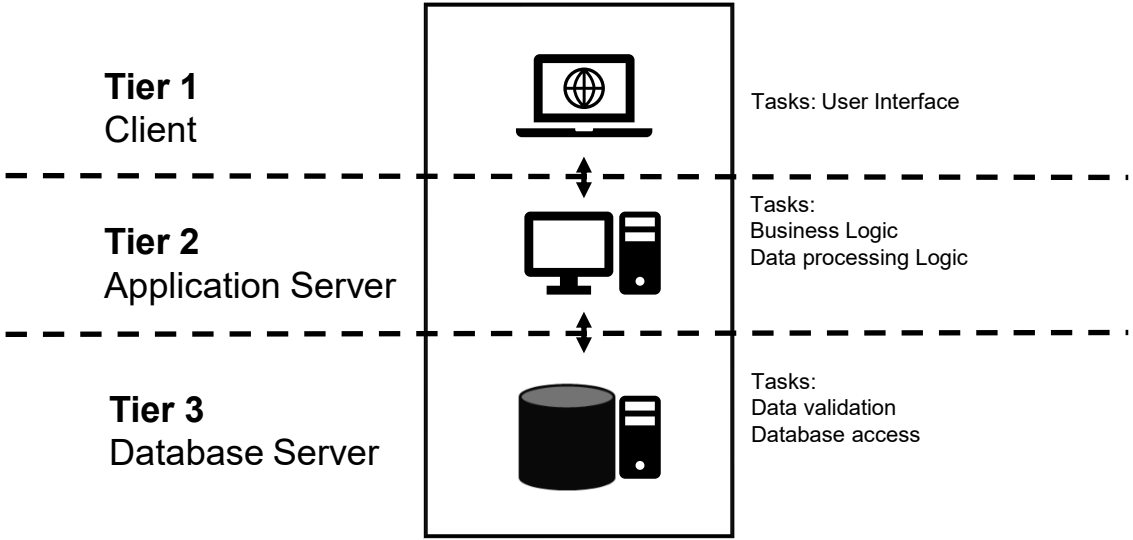
- **CREATE DATABASE** - creates a new database
- **ALTER DATABASE** - modifies a database
- **CREATE TABLE** - creates a new table
- **ALTER TABLE** - modifies a table
- **DROP TABLE** - deletes a table
- **CREATE INDEX** - creates an index (search key)
- **DROP INDEX** - deletes an index

DDL

- **USE DATABASE** – uses a particular database
- **SELECT** - extracts data from a database
- **UPDATE** - updates data in a database
- **DELETE** - deletes data from a database
- **INSERT INTO** - inserts new data into a database

DML

3-Tier Client-Server Architecture



CB – GE – Chapter 3, section 3.1

Distributed or Single machine?

Imagine if there are two or more booking staff terminals – how will you manage data consistency and/or synchronization?

Data Access with Python

- In order to use a specific DBMS, the corresponding libraries should be installed.
  - For example, for MySQL, mysql-connector-python library can be installed
    - `pip install mysql-connector-python`
  - We will practice SQLite library version 3, sqlite3; and mysql-connector-python for MySQL

[MUST READ] What should you choose for your group project? MySQL vs. SQLite vs.PostgreSQL? Must Read:
https://www.digitalocean.com/community/tutorials/sqlite-vs-mysql-vs-postgresql-a-comparison-of-relational-database-management-systems

Data management with sqlite3

```
import sqlite3
con = sqlite3.connect('example.db')
cur = con.cursor()
```

Importing module and creating the objects

```
# Create table
cur.execute('''CREATE TABLE stocks
            (date text, trans text, symbol text, qty real, price real)''')
```

Table created

```
# Insert a row of data
cur.execute("INSERT INTO stocks VALUES ('2006-01-05','BUY','RHAT',100,35.14)")
```

Data inserted

```
# Save (commit) the changes
con.commit()
```

Changes upon data recorded in DB

```
# We can also close the connection if we are done with it.
# Just be sure any changes have been committed or they will be lost.
con.close()
```

Connection closed

sqlite_1.py Example

Data management with sqlite3

```
lst = [('2006-01-05', 'BUY', 'RHAT', 100, 35.14),
       ('2006-03-28', 'BUY', 'IBM', 1000, 45.0),
       ('2006-04-06', 'SELL', 'IBM', 500, 53.0),
       ('2006-04-05', 'BUY', 'MSFT', 1000, 72.0),
]
cur.executemany("insert into stocks values (?,?,?,?,?)",lst)
```

Collective insert

```
for row in cur.execute("SELECT * FROM stocks ORDER BY price"):
    print(row)
```

Select many and display

```
#Delete
cursor = conn.cursor()
cursor.execute("delete from project where name = 'virtualenvwrapper'")
```

Delete a given record

```
cursor = conn.cursor()
query = "update task set status = :status where id = :id"
cursor.execute(query, {'status':status, 'id':id})
```

Update a given record

DB Browser for SQLite
https://sqlitebrowser.org/

MySQL (ver 4.2.0 or older)
Extension for VS Code

Sqlite_2.py Example

Running SQL script in Python

```python
import os
import sqlite3

db_filename = 'myDB.db'
schema_filename = 'users.sql'

#db_is_new = not os.path.exists(db_filename)
db_is_new = True

with sqlite3.connect(db_filename) as conn:
    if db_is_new:
        print ('Creating schema')
        with open(schema_filename, 'rt') as f:
            schema = f.read()
        conn.executescript(schema)
```

Sqlite_4.py Example

users.sql

```sql
create table user (
    uname       text primary key,
    passwd      text,
    utype       text
);

create table details (
    id          integer primary key autoincrement not null,
    name        text,
    address     text,
    email       text,
    user        text not null references user(uname)
);
```

- An existing .sql file can be executed within Python code using sqlite3 functionalities. The rest of the .py file is to insert more data in.

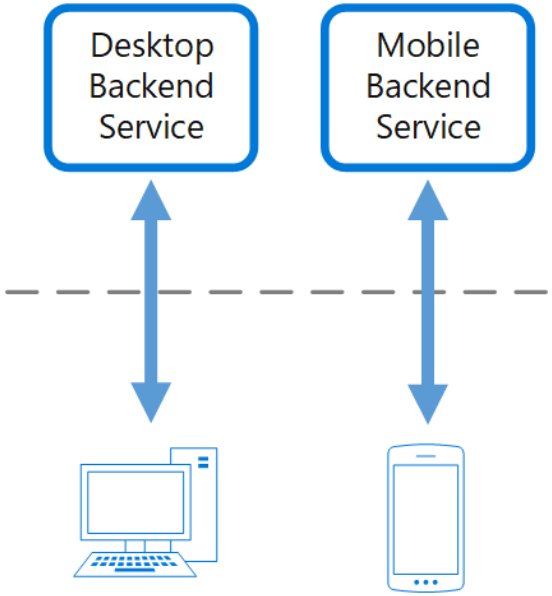
Running SQL script in Python

```
conn.execute("insert into user (uname, passwd, utype) \
        values ('meaydin', 'mypasswd01', 'user')")

conn.execute("insert into details (name, address, email, user) \
        values ('Mehmet Aydin',  '100 Ford St Bath',   'mehmet.aydin@uwe.ac.uk', 'meaydin') ")

conn.execute("insert into user (uname, passwd, utype) \
    values ('arakib', 'hispasswd01', 'admin')")

conn.execute("insert into details (name, address, email, user) \
    values ('Abdur Rakib', 'Coldharbour Lane, Bristol', 'rakib.abdur@uwe.ac.uk', 'arakib') ")

conn.commit()
```

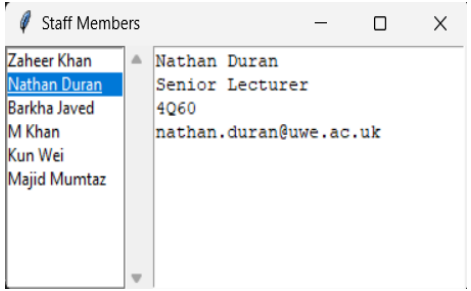
Merging GUI with DB operations

- GUI programming facilitates developing UI/frontend of an application, while DB operations are used to build backend components.

- Let's revisit Staff.py GUI to let interact with a database so as to pull and push data from/to DB tables.



Staff App Details Example (from last Session)

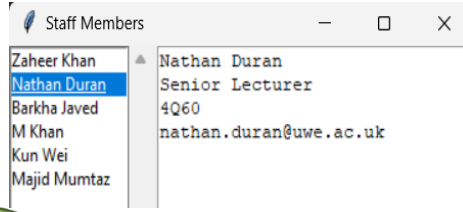



```python
from tkinter import *
import json

window=Tk()
window.title("Staff Members")

lstName=Listbox(window,width=100,height=100)
yscroll=Scrollbar(command=lstName.yview)
yscroll.grid(row=0,column=3,sticky=NS,rowspan=3)

lstName=Listbox(window,width=15,height=10,yscrollcommand=yscroll.set)
lstName.grid(row=0,padx=0,pady=0,sticky=W,columnspan=2,rowspan=3)

yscroll.config(command=lstName.yview)

txt = Text(window,width=30,height=10)
txt.grid(padx=5,pady=0,row=0,column=5)
```

```python
.....

with open("users.json",'r') as l:
    loaded_list = json.load(l)
    for line in loaded_list:
        countryName=line[0]
        lstName.insert('end',countryName)

def select(evt):
    value = lstName.curselection()[0]
    templist = loaded_list[value]

    txt.delete(1.0,END)
    for i in templist:
        txt.insert(END,i+'\n')

lstName.bind('<<ListboxSelect>>',select)

window.mainloop()
```

Staff_1.py Example

Staff Details Example (Revised)



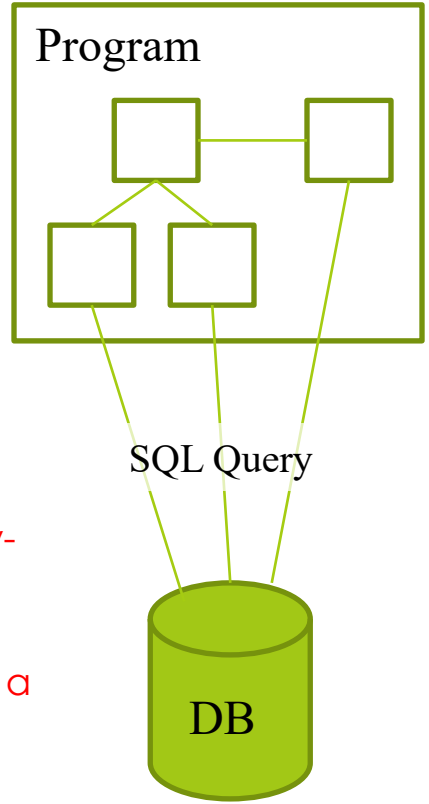

```python
from tkinter import *
#import json
import sqlite3

def upload():
    con = sqlite3.connect('myDB.db')
    cur = con.cursor()

    cur.execute("SELECT * FROM staff ORDER BY name")
    lst = cur.fetchall()
    con.close()

    for row in lst:
        print(row)

    return lst
```

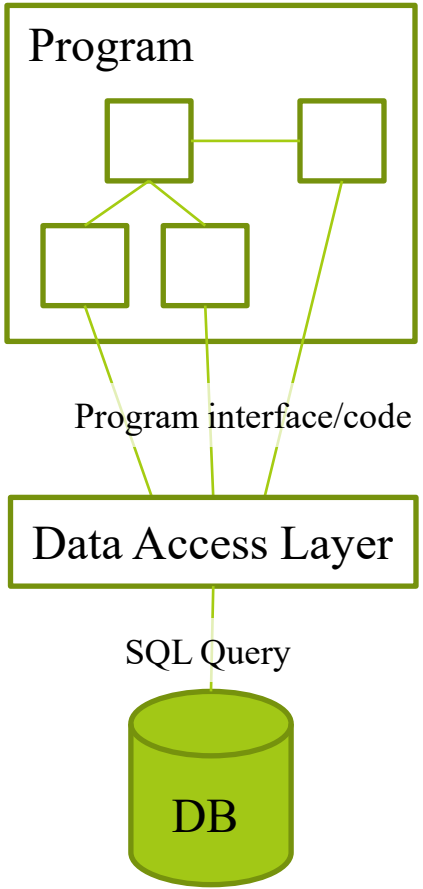
```python
window=Tk()
window.title("Staff Members")

lstName=Listbox(window,width=100,height=100)
yscroll=Scrollbar(command=lstName.yview)
yscroll.grid(row=0,column=3,sticky=NS,rowspan=3)
```

```python
.....

loaded_list = upload()
for line in loaded_list:
    countryName=line[0]
    lstName.insert('end',countryName)


def select(evt):
    value = lstName.curselection()[0]
    templist = loaded_list[value]

    txt.delete(1.0,END)
    for i in templist:
        txt.insert(END,i+'\n')

lstName.bind('<<ListboxSelect>>',select)

window.mainloop()
```

Staff_1.py Example

DB Access





- In general, distributing low-level data access logic throughout a program is <u>not</u> a good idea (design).

Data Access Layer

- A better design is one that includes a data access layer which **encapsulates** the details of the underlying persistence API.
- It **abstracts** the low-level details of persistent storage.
- It provides an **interface** that is usually a better match for the style of programming used in the domain logic. For example, the data access layer might provide an **OO interface onto relational data**.

Study Data Access Design Patterns:
https://martinfowler.com/eaaCatalog/index.html

Should we create a **Data Access Object (DAO)**?

SQLAlchemy is an object-oriented approach of Python to deal with relational data, where it uses ORM (object relational mapper), which can be viewed as kind of DAO

Wouldn't it be more appropriate to have object-oriented database management systems?

Object Oriented Databases

Read Chapter 27 and 28 from CB-GE and
Chapter 12 from AN-GE

- **CB-GE:** Database Systems: practical. Approach to design, implementation, and management, Connolly and Begg (2015)
- **AN-GE:** Fundamentals of Database Systems, Ramez Elmasri (2017)

Object Oriented Databases

- Persistence storage of objects
- Example: Realm (mongoDB), DB2, PostgreSQL (Object RDBMS)
- Must read: https://www.mongodb.com/databases/what-is-an-object-oriented-database
- Objects with Relational DB
  - Mapping Classes to Relational tables - https://sparxsystems.com/resources/tutorials/uml/datamodel.html
  - About storing objects as columns or Rows – Oracle
    - https://docs.oracle.com/en/database/oracle/oracle-database/19/adobj/general-storage-considerations-for-objects.html

Classes (OO) to Tables (Relational)

- MUST READ:
  - Class Model and Relational Model
  - Suggestive process to map Class Model to the Relational Model

https://sparxsystems.com/resources/tutorials/uml/datamodel.html

Process:

- Model classes,
- Identify persistent objects,
- Assume each persistent class maps to one relational table,
- select an inheritance strategy:
  - Each class hierarchy has single table with all the inherited attributes for all elements
  - Each class in the hierarchy has a corresponding table of only the attributes accessible by that class (including inherited attributes).
  - Each generation in the class hierarchy has a table containing only that generation's actual attributes.

- For each class add a unique object identifier
- Map attributes to columns
- Map associations to foreign keys
- Map aggregation and composition
- Define relationship roles
- Model behaviour (trigger, stored procedures)
- Produce a physical model

ORM – Object Relational Mapper

Object Relational Mapper (ORM):

- Programming technique for converting data between incompatible type systems in OOP languages
- Remove the necessity of writing tedious and error-prone raw SQL statements

- Provides **data mapper pattern**, where classes can be mapped to the database in open ended, multiple ways – allowing the object model and database schema to develop in a cleanly decoupled way from the beginning.

SQLAlchemy is Python SQL Toolkit

https://www.sqlalchemy.org/

SQL Alchemy

Now with **SQLAlchemy**, there are three most important components:

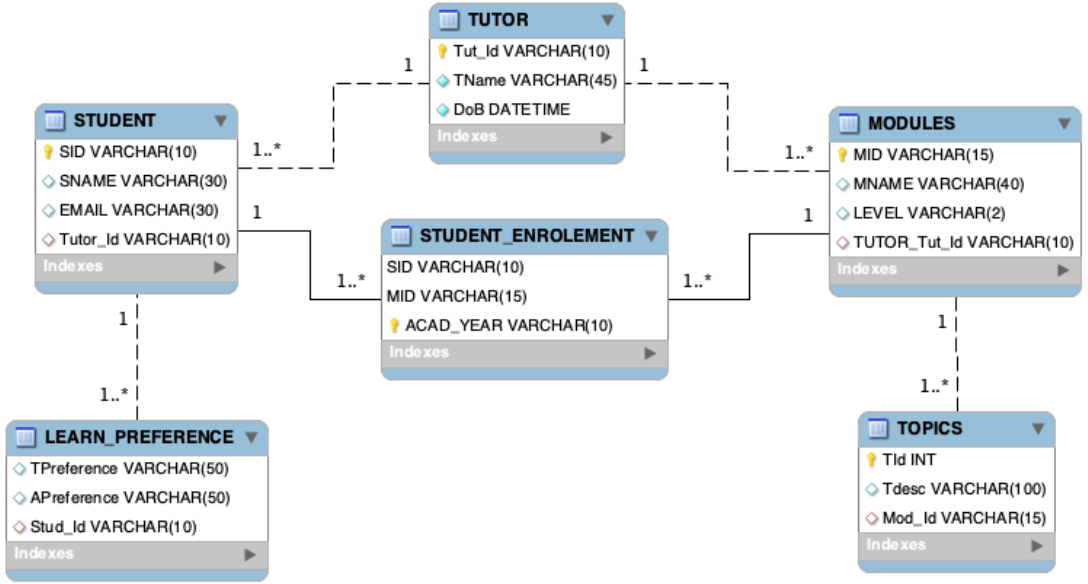
- A **Table** that represents a table in a database
- A **mapper** that maps a Python Class to a table in a database.
- A **class object** that defines how a database record maps to a normal Python object.

SQLAlchemy's declarative allows to define a Table, a mapper and a class object in one class definition

#EXAMPLE ON NEXT SLIDES IS ADAPTED FROM https://www.pythoncentral.io/introductory-tutorial-python-sqlalchemy/

But before that… let's take an example that you're familiar with…

This is what we learned – How to write MySQL database code in Python…. without using an ORM



Example..

mysql.connector

pip install python-mysql-connector==9.6.0

```python
import mysql.connector, dbfunc
conn = dbfunc.getConnection()   #connection to DB
DB_NAME = 'studentreg'              #DB Name
TABLE_NAME = 'student'
tid = input('Enter TempID: ')
# here you should perform data validation syntax/semantics
#DELETE_statement = 'SELECT * FROM ' + TABLE_NAME + ' \
#    WHERE tempId = ' + tid + ';'   #remember SQL Injection

SELECT_statement = 'SELECT * FROM ' + TABLE_NAME + ' \
    WHERE SID = %s;'

if conn != None:    #Checking if connection is None
    if conn.is_connected(): #Checking if connection is established
        print('MySQL Connection is established')
        dbcursor = conn.cursor()    #Creating cursor object
        dbcursor.execute('USE {};'.format(DB_NAME)) #use database
        dataset = (tid,)
        dbcursor.execute(SELECT_statement, dataset)
        print('SELECT statement executed successfully.')
        row = dbcursor.fetchone()
        while row is not None:
            print(row, '\n')
            row = dbcursor.fetchone()
        dbcursor.close()
        conn.close() #Connection must be closed
    else:
        print('DB connection error')
else:
    print('DBFunc error')
```

PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

```
Material/week10-new/WDaDB - MySQL python - examples/simpleSELECT.py"
Enter TempID: 1002
MySQL Connection is established
SELECT statement executed successfully.
('1002', 'Jack Davis', 'jack.davis@abc.com', '1001')
```

Example..

tutor

name	Type	Nullable
id	int	No
name	String(250)	No

modules

name	Type	Nullable
id	int	No
module_name	String(100)	Yes
module_desc	String(250)	Yes
module_code	String(100)	No
tutor_id	int	No

SQLite

Example…

```python
import sqlite3
conn = sqlite3.connect('example.db')

c = conn.cursor()
c.execute('''
CREATE TABLE tutor
(id INTEGER PRIMARY KEY ASC, name varchar(250) NOT NULL)
''')
c.execute('''
CREATE TABLE modules
(id INTEGER PRIMARY KEY ASC, module_name varchar(100), module_desc varchar(250),
module_code varchar(100) NOT NULL, tutor_id INTEGER NOT NULL,
FOREIGN KEY(tutor_id) REFERENCES tutor(id))
''')
c.execute('''
INSERT INTO tutor VALUES(1, 'Zaheer Khan')
''')
c.execute('''
INSERT INTO tutor VALUES(2, 'James Lear')
''')
c.execute('''
INSERT INTO modules VALUES(1, 'Web Development and Databases',
    'Teaches web and database concepts', 'UFCFES-30-1', 1)
''')
c.execute('''
INSERT INTO modules VALUES(2, 'Advanced Software Development',
    'Teaches SDLC through Object-Oriented Analysis and Design', 'UFCF8S-30-2', 1)
''')
c.execute('''
INSERT INTO modules VALUES(3, 'Object Oriented Analysis and Design',
    'Teaches OO Analysis and Design', 'UFCFXS-30-2', 1)
''')
```

```python
c.execute('SELECT * FROM tutor')
print(c.fetchall())
c.execute('SELECT * FROM modules')
print(c.fetchall())
conn.commit()
conn.close()
```

```
PS C:\Users\m3-mumtaz\OneDrive - UWE Bristol\Spring_Modules\ASD TB2\Teaching Block 2 Material\week10-new\sqlalchemyexamples> python .\0_convent
al_db.py
[(1, 'Zaheer Khan'), (2, 'James Lear')]
[(1, 'Web Development and Databases', 'Teaches web and database concepts', 'UFCFES-30-1', 1), (2, 'Advanced Software Development', 'Teaches SDL
hrough Object-Oriented Analysis and Design', 'UFCF8S-30-2', 1), (3, 'Object Oriented Analysis and Design', 'Teaches OO Analysis and Design', 'U
XS-30-2', 1)]
```

SQL Alchemy Example

Install the module:
pip install sqlalchemy

Creating Model/Tables

Classes for tables

engine = create_engine("mysql+mysqlconnector://user:password@host/database")

Relationship

Engine to store data

Create tables

```python
from sqlalchemy import Column, ForeignKey, Integer, String
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import relationship
from sqlalchemy import create_engine

Base = declarative_base()

class Tutor(Base):
    __tablename__ = 'tutor'
    #define columns for table tutor where each column is
    # a normal Python instance attribute
    id = Column(Integer, primary_key=True)
    name = Column(String(250), nullable=False)

class Modules(Base):
    __tablename__ = 'modules'
    #define columns for table modules where each column is
    # a normal Python instance attribute
    id = Column(Integer, primary_key=True)
    module_name = Column(String(100))
    module_desc = Column(String(250))
    module_code = Column(String(100), nullable=False)
    tutor_id = Column(Integer, ForeignKey('tutor.id'))
    tutor = relationship(Tutor)
# Create an engine that stores data in the local
# sqlachemy_db.db file
engine = create_engine('sqlite:///sqlalchemy_db.db')

# Create all tables in the engine - similar to Create Table
Base.metadata.create_all(engine)
```

SQL Alchemy

INSERT Records in the DB

```python
#Adding records
#Source: https://www.pythoncentral.io/introductory-tutorial-python-sqlalchemy/
from sqlachemy_declarative import Modules
from sqlachemy_declarative import Tutor
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

from sqlachemy_declarative import Tutor, Base, Modules

engine = create_engine('sqlite:///sqlalchemy_db.db')
# Bind the engine to the metadata of the Base class so that the
# declaratives can be accessed through a DBSession instance
Base.metadata.bind = engine

DBSession = sessionmaker(bind=engine)
# A DBSession() instance establishes all conversations with the datal
# and represents a "staging zone" for all the objects loaded into th
# database session object. Any change made against the objects in th
# session won't be persisted into the database until you call
# session.commit(). If you're not happy about the changes, you can
# revert all of them back to the last commit by calling
# session.rollback()
session = DBSession()

# Insert a Tutor in the tutor table
new_tutor = Tutor(id=10099, name='Zaheer Khan')
session.add(new_tutor)
session.commit()
```

```python
# Insert a Module in the module table
# 'Web Development and Databases', 'Teaches web and database concepts', 'UFCFES-30-1', 1
new_module = Modules(id=1000, module_name= 'Web Development and Databases',
                     module_desc='Teaches web and database concepts',
                     module_code='UFCFES-30-1', tutor=new_tutor)
session.add(new_module)
session.commit()

#'Advanced Software Development','Teaches SDLC through Object-Oriented Analysis and Design', 'UFCF8S-30-2'
new_module = Modules(id=1001, module_name= 'Advanced Software Development',
                     module_desc='Teaches SDLC through Object-Oriented Analysis and Design',
                     module_code='UFCF8S-30-2', tutor=new_tutor)
session.add(new_module)
session.commit()

new_tutor = Tutor(id=10001, name='James Lear')
session.add(new_tutor)
session.commit()

# 'Object Oriented Analysis and Design', 'Teaches OO Analysis and Design', 'UFCFXS-30-2'
new_module = Modules(id=1002, module_name= 'Object Oriented Analysis and Design',
                     module_desc='Teaches OO Analysis and Design',
                     module_code='UFCFXS-30-2', tutor=new_tutor)
session.add(new_module)
session.commit()
```

Continue to next page…

```python
# Insert a Module in the module table
# 'Web Development and Databases', 'Teaches web and database concepts', 'UFCFES-30-1', 1
new_module = Modules(id=1000, module_name= 'Web Development and Databases',
                     module_desc='Teaches web and database concepts',
                     module_code='UFCFES-30-1', tutor=new_tutor)
session.add(new_module)
session.commit()

#'Advanced Software Development','Teaches SDLC through Object-Oriented Analysis and Design', 'UFCF8S-30-2'
new_module = Modules(id=1001, module_name= 'Advanced Software Development',
                     module_desc='Teaches SDLC through Object-Oriented Analysis and Design',
                     module_code='UFCF8S-30-2', tutor=new_tutor)
session.add(new_module)
session.commit()

new_tutor = Tutor(id=10001, name='James Lear')
session.add(new_tutor)
session.commit()

# 'Object Oriented Analysis and Design', 'Teaches OO Analysis and Design', 'UFCFXS-30-2'
new_module = Modules(id=1002, module_name= 'Object Oriented Analysis and Design',
                     module_desc='Teaches OO Analysis and Design',
                     module_code='UFCFXS-30-2', tutor=new_tutor)
session.add(new_module)
session.commit()
```

INSERT SQL Alchemy

SQL Alchemy QUERY

```
from sqlachemy_declarative import Modules
from sqlachemy_declarative import Tutor
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

from sqlachemy_declarative import Tutor, Base, Modules

engine = create_engine('sqlite:///sqlalchemy_db.db')
# Bind the engine to the metadata of the Base class so that the
# declaratives can be accessed through a DBSession instance
Base.metadata.bind = engine
DBSession = sessionmaker(bind=engine)
session = DBSession()
```

SQL Alchemy QUERY

```
print('\n# Find all tutors\n')
for tutor in session.query(Tutor).all():
    print(str(tutor.id) + ' ' + tutor.name)

print('\n# Find first tutor\n')
tutor = session.query(Tutor).first()
print(tutor.name)

print('\n#find modules matching above tutor\n')
modules = session.query(Modules).filter(Modules.tutor == tutor)
for module in modules:
    print(module.id, module.module_name, module.module_desc, module.module_code)
```

Continue to next page…

SQL Alchemy QUERY

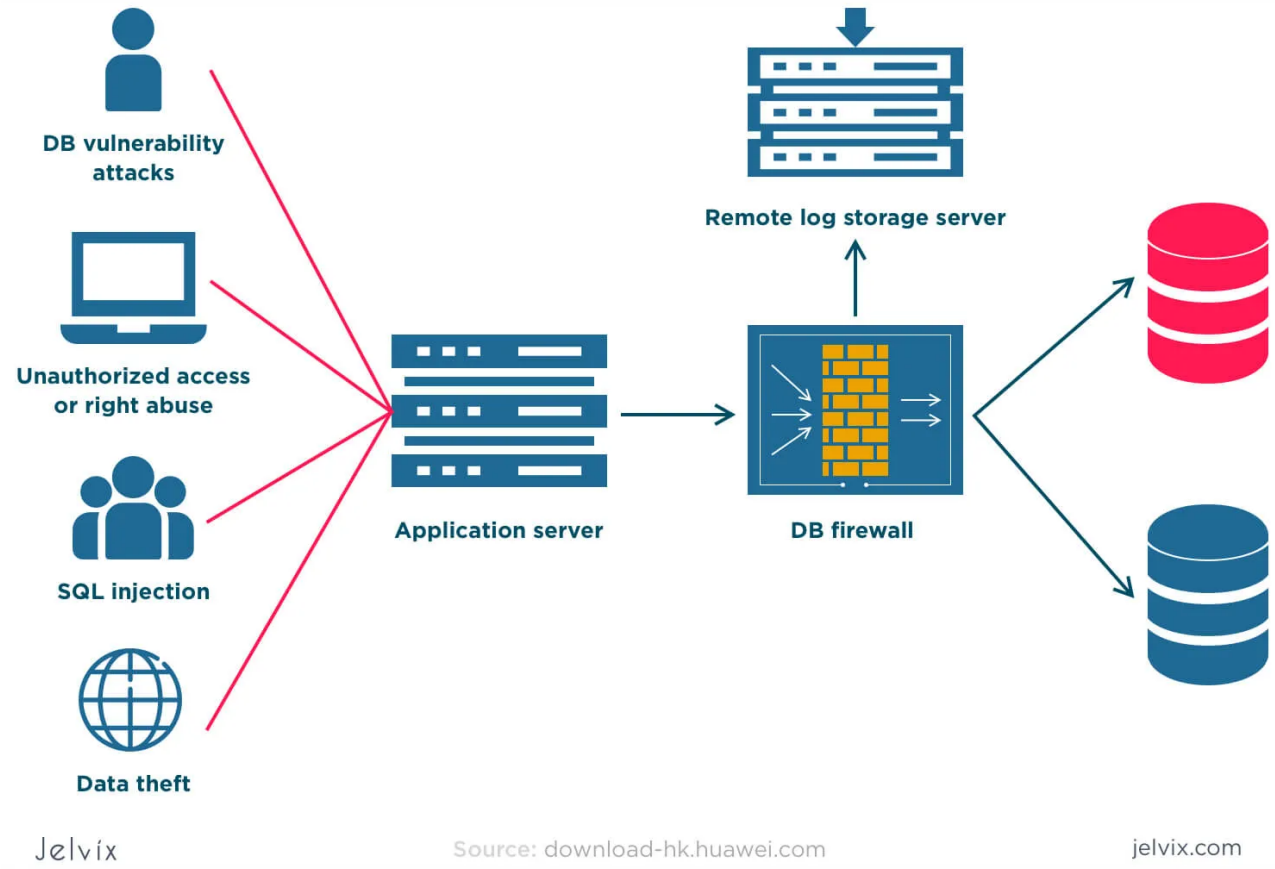
```
print('\n#join\n')
result = session.query(Tutor, Modules).join(Modules).all()

for tutor, modules in result:
    print(tutor.id, tutor.name, modules.module_name, modules.module_desc, modules.module_code)

print('\n#join with filter and selection in query\n')
result = session.query(Tutor.id, Tutor.name).join(Modules).filter(Modules.module_code=='UFCFES-30-1').all()
for row in result:
    print(row) #will show tutor id and tutor name

print('\n#join with a filter and selection in print\n')
result = session.query(Tutor, Modules).join(Modules).filter(Tutor.id==1000).all()
for tutor, modules in result:
    print(tutor.id, tutor.name, modules.module_name, modules.module_desc, modules.module_code)

#if three table join then add select_from
#result = session.query(Tutor, Modules, Topics).select_from(Topics).join(Modules).join(Tutor).all()
#for topics, modules, tutor in result:
#    print(topic.name, tutor.name, modules.module_name)
```

SQL Alchemy

Output on Terminal

```
# Find all tutors

1000 Zaheer Khan
1001 James Lear
1009 Eseosa
10099 Zaheer Khan

# Find first tutor

Zaheer Khan

#find modules matching above tutor

1000 Web Development and Databases Teaches web and database concepts UFCFES-30-1
1001 Advanced Software Development Teaches SDLC through Object-Oriented Analysis and Design UFCF8S-30-2

#join

1000 Zaheer Khan Web Development and Databases Teaches web and database concepts UFCFES-30-1
1000 Zaheer Khan Advanced Software Development Teaches SDLC through Object-Oriented Analysis and Design UFCF8S-30-2
1001 James Lear Object Oriented Analysis and Design Teaches OO Analysis and Design UFCFXS-30-2

#join with filter and selection in query

(1000, 'Zaheer Khan')

#join with a filter and selection in print

1000 Zaheer Khan Web Development and Databases Teaches web and database concepts UFCFES-30-1
1000 Zaheer Khan Advanced Software Development Teaches SDLC through Object-Oriented Analysis and Design UFCF8S-30-2
```

Database Security



Key Aspects of Database Security

1) Access Control
2) Authentication and Encryption
3) Regular audit and monitoring
4) Backup and Recovery

Distributed Databases

- Parts of database are stored on multiple physical locations and processing is distributed among multiple database nodes
- Two or more files located in different locations (same/different network)
- A centralized distributed database management system integrates data logically to provide seamless access (i.e. as it is stored in the same location)
  - DDBMS synchronises all the data periodically to ensure data updates/deletes performed at one location are automatically reflected in the data stored elsewhere

Read Chapter 24 and 25 from CB-GE and Chapter 23 from EN-GE

Distributed databases article

https://www.tutorialspoint.com/distributed_dbms/distributed_dbms_quick_guide.htm

- Horizontal fragmentation
- Vertical fragmentation
- Examples: Apache Cassandra, Apache HBase, Couchbase Server, Amazon SimpleDB, Apache Ignite, FoundationDB

Booking

BookingID	RoomID	GuestID	CheckInDate	CheckOutDate	NumGuests
B100	R201	G501	2023-10-27	2023-10-29	2
B101	R202	G502	2023-10-27	2023-10-30	1
B102	R203	G503	2023-10-28	2023-10-31	3
B103	R201	G504	2023-10-28	2023-10-30	2
B104	R204	G505	2023-10-29	2023-11-01	4
B105	R202	G506	2023-10-29	2023-10-31	1
B106	R205	G507	2023-10-30	2023-11-02	2
B107	R203	G508	2023-10-30	2023-11-01	3

Room: R201

BookingID	GuestID	CheckInDate	CheckOutDate	NumGuests	TotalPrice
B100	G501	2023-10-27	2023-10-29	2	$300
B103	G504	2023-10-28	2023-10-30	2	$350

Room: R202

BookingID	GuestID	CheckInDate	CheckOutDate	NumGuests	TotalPrice
B101	G502	2023-10-27	2023-10-30	1	$450
B105	G506	2023-10-29	2023-10-31	1	$280

Booking

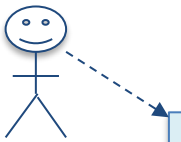
BookingID	Reservation#	Covers	ReservationDate	TableType	SpecialRequests	CustomerID	RestaurantID

BookingID	Reservation#	Covers

BookingID	TableType	SpecialRequests	CustomerID	RestaurantID

Distributed databases article

MVC Design Pattern



User interface
GUI, Tkinter
window, widgets

Business Logic

Data
Data handling
Data validation/integrity

View ↔ Controller → Model

Model ↕ Storage

Little logic to display data in the model
Communicates with controller
No direct communication with model

Bridges the gap between model and view

Routes data between view and models

Model must not depend on the view and controller

MVC StaffApp version

Model

1. Create Model
2. Create View
3. Create Controller
4. Create App
5. Let App connect Controller with View and Model

```python

def delete_data(self, name):
    try:
        staff = self.session.query(Staff).filter_by(name=name).first()
        if staff:
            self.session.delete(staff)
            self.session.commit()
            messagebox.showinfo("Success", "Staff member deleted successfully.")
        else:
            messagebox.showerror("Error", "Staff member not found.")
    except Exception as e:
        self.session.rollback()
        messagebox.showerror("Database Error", f"Error deleting data: {e}")
```

```python
# Model
class StaffModel:
    def __init__(self, db_name='sqlite:///myDB.db'):
        self.engine = create_engine(db_name)
        Base.metadata.create_all(self.engine)
        Session = sessionmaker(bind=self.engine)
        self.session = Session()

    def upload(self):
        try:
            return self.session.query(Staff).order_by(Staff.name).all()
        except Exception as e:
            messagebox.showerror("Database Error", f"Error fetching data: {e}")
            return []

    def insert_data(self, lst):
        try:
            staff = Staff(
                name=lst[0],
                position=lst[1],
                office=lst[2],
                email=lst[3]
            )
            self.session.add(staff)
            self.session.commit()
            messagebox.showinfo("Success", "Staff member added successfully.")
        except Exception as e:
            self.session.rollback()
            messagebox.showerror("Database Error", f"Error inserting data: {e}")
```

MVC StaffApp version

View

```python
# View
class StaffView:
    def __init__(self, master, controller):
        self.master = master
        self.master.title("Staff Members")
        self.controller = controller

        self.uName = StringVar()
        self.uEmail = StringVar()
        self.uRoom = StringVar()
        self.uPos = StringVar()

        # Listbox with scrollbar
        self.lstName = Listbox(self.master, width=15, height=10)
        yscroll = Scrollbar(command=self.lstName.yview)
        yscroll.grid(row=0, column=2, sticky=NS, rowspan=3)
        self.lstName.config(yscrollcommand=yscroll.set)
        self.lstName.grid(row=0, padx=0, pady=0, sticky=W, columnspan=2, rowspan=3)

        # Details text area
        self.txt = Text(self.master, width=30, height=10)
        self.txt.grid(padx=5, pady=0, row=0, column=5)

        # Buttons frame
        f = Frame(self.master)
        f.grid(row=1, column=5)
        Button(f, text="Add", command=self.controller.addNew).grid(row=1, column=0, sticky=W)
        Button(f, text="Remove", command=self.controller.remove).grid(row=1, column=1, sticky=E)

        self.lstName.bind('<<ListboxSelect>>', self.controller.select)

    def populate_listbox(self, data):
        self.lstName.delete(0, END)
        for staff in data:
            self.lstName.insert(END, staff.name)
```

MVC StaffApp version

Controller

staffAppMVC_OODB.py > ...

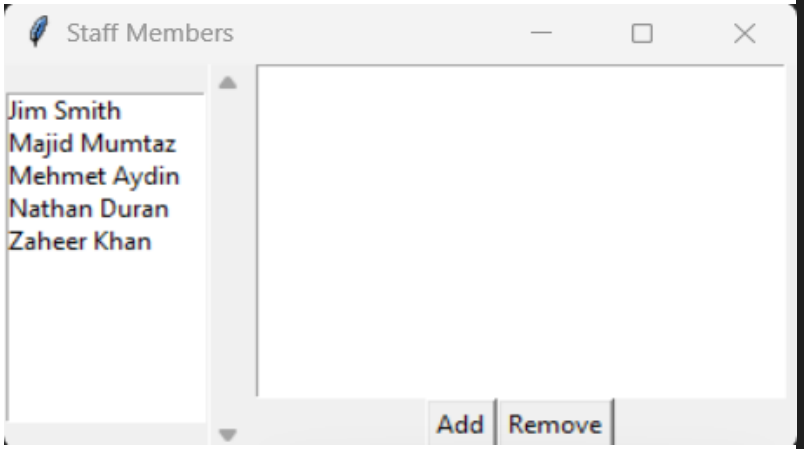
```python
# Controller
class StaffController:
    def __init__(self, master):
        self.master = master
        self.model = StaffModel()
        self.view = StaffView(master, self)
        self.loaded_list = self.model.upload()
        self.view.populate_listbox(self.loaded_list)

    def select(self, evt):
        try:
            index = self.view.lstName.curselection()[0]
            selected_staff = self.loaded_list[index]
            self.view.populate_text(selected_staff)
        except IndexError:
            pass

    def register(self):
        new_staff = [
            self.view.uName.get(),
            self.view.uPos.get(),
            self.view.uRoom.get(),
            self.view.uEmail.get()
        ]
        self.model.insert_data(new_staff)
        self.loade
        self.view.populate_listbox(self.loaded_list)
```

(method) def populate_listbox(data: Any) -> None

Simplified version

Application

Output




```python
    def addNew(self):
        nMem = Toplevel(self.master)
        nMem.title("Registration")
        nMem.geometry("210x220")
        nMem.resizable(False, False)

        labelframe = LabelFrame(nMem, text="Add New Staff Member!")
        labelframe.grid(row=0, column=0, columnspan=2, padx=5, pady=5) # Added grid t

        Label(labelframe, text="Staff Name:").grid(row=0, column=0)
        Entry(labelframe, textvariable=self.view.uName).grid(row=0, column=1)
        Label(labelframe, text="Position:").grid(row=1, column=0)
        Entry(labelframe, textvariable=self.view.uPos).grid(row=1, column=1)
        Label(labelframe, text="Room:").grid(row=2, column=0)
        Entry(labelframe, textvariable=self.view.uRoom).grid(row=2, column=1)
        Label(labelframe, text="Email:").grid(row=3, column=0)
        Entry(labelframe, textvariable=self.view.uEmail).grid(row=3, column=1)

        Button(nMem, text="Register", command=self.register).grid(row=4, column=0)
        Button(nMem, text="Close", command=nMem.destroy).grid(row=4, column=1)

    def remove(self):
        try:
            index = self.view.lstName.curselection()[0]
            staff = self.loaded_list[index]
            self.model.delete_data(staff.name)
            self.loaded_list = self.model.upload()  # Refresh from database
            self.view.populate_listbox(self.loaded_list)
            self.view.clear_text()
        except IndexError:
            messagebox.showerror("Error", "Please select a staff member to remove.")

if __name__ == "__main__":
    root = Tk()
    controller = StaffController(root)
    root.mainloop()
```

MVC Demo

Summary

- There is lot to learn about Databases and it's good that you started databases last year
- Databases are important to manage persistent data
- Design decisions play critical role in selecting file-based storage or DB
- Selecting a database management system is also critical and depends on the type of project
- Programming languages support through middleware or libraries
  - Python libraries for sqlite3 and python-mysql-connector
- OO classes to relational tables approach Vs Object Oriented Databases
  - Object Relational Mapper e.g., SQLAlchemy
- Distributed Databases
- Model-View Controller promotes reusability and helps in managing complexity of expanding application
- **Next week** – we'll start with Software testing and LESP Issues.

References

- Database Systems: A practical approach to design, implementation, and management, Thomas Connolly and Carolyn Begg. Sixth Edition/Global Edition. (CB-GE)
- Fundamentals of Database Systems, Ramez Elmasri and Shamkant B Navathe. Seventh Edition/Global Edition. (EN-GE)
- Real Python Tutorial
- Python Databases Tutorials
- Popular Relational DBMS **https://www.digitalocean.com/community/tutorials/sqlite-vs-mysql-vs-postgresql-a-comparison-of-relational-database-management-systems**
- Classes to Relational Tables - https://sparxsystems.com/resources/tutorials/uml/datamodel.html
- Distributed databases article
- Distributed Databases: https://www.tutorialspoint.com/distributed_dbms/distributed_dbms_quick_guide.htm
- https://martinfowler.com/eaaCatalog/index.html
- SQLAlchemy: https://www.sqlalchemy.org/
  - https://www.pythoncentral.io/introductory-tutorial-python-sqlalchemy/